

LAB TWO –Single Segment

In this lab, you will learn how to use Wireshark, a software package used to monitor link activity. You will also learn about ARP and more extensive commands on how to configure the PCs. All the lab exercises use the network configuration shown in Figure 2.1.

Connect all four PCs (PC1 - PC4) to a single Ethernet segment via a single hub as shown in Figure 2.1. Configure the IP addresses for the PCs as shown in Table 2.1.

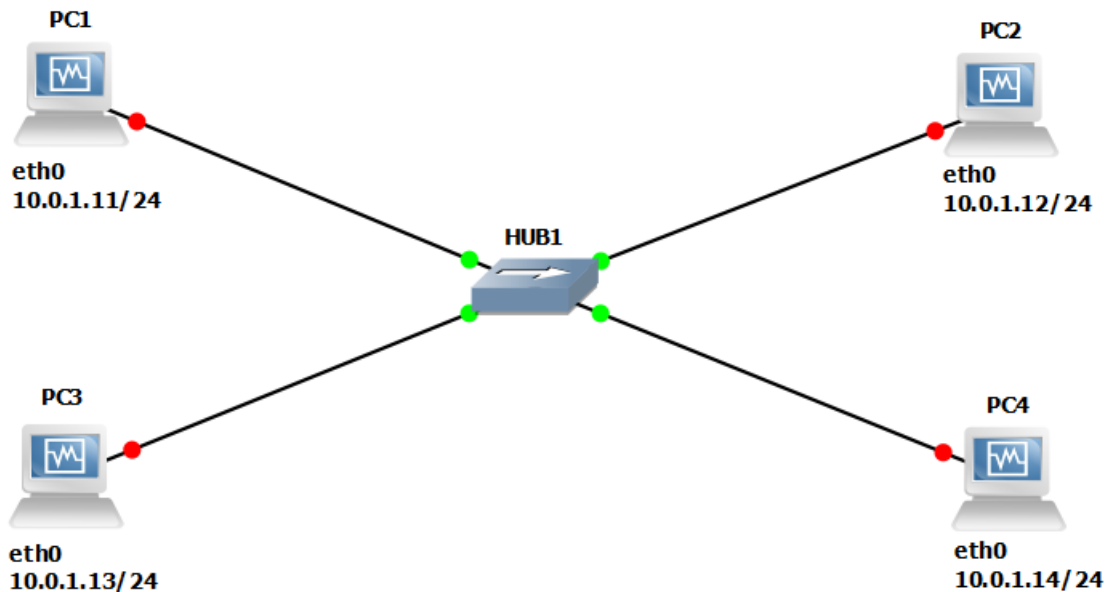


Figure 2.1 - Network Configuration for Lab 2.

PCs	IP Addresses of Ethernet Interface eth0
PC1	10.0.1.11 / 24
PC2	10.0.1.12 / 24
PC3	10.0.1.13 / 24
PC4	10.0.1.14 / 24

Table 2.1 - IP Addresses for Figure 2.1



Tip: Recall the following Linux command from Lab 1, Part 3, that was used to set up the IP addresses.

```
ifconfig interface_name IPAddress/XX
```

PART 1. Starting WIRESHARK

Download the Wireshark software from [here](#) for both Windows and Macs.

PART 2. Capturing Traffic using WIRESHARK

In this part of the lab, you experiment with filter expressions within the Wireshark application. The filtering capabilities and options of Wireshark are described under the help tab in Wireshark.

Exercise 1. Display filters and traffic capture with Wireshark

1. Configure the network topology as shown in Figure 2.1 above and configure the PC IP addresses with the values shown in Table 2.1 using the `ifconfig` command introduced in Lab 1.
2. Right click on the link that connects PC1 and the Ethernet Hub and select "Start capture".
NOTE: Please choose the **HUB side of the link for capturing, not the PC side**. Choose option **Ethernet** (only option available). Then right click on the icon and you will see "Start Wireshark". The Wireshark Network Analyzer then opens in a new window.
 - a) Please make sure that the Packet Capture settings are set to Wireshark Live Traffic Capture from the "Packet capture preferences" tab as shown below in Figure 2.2.

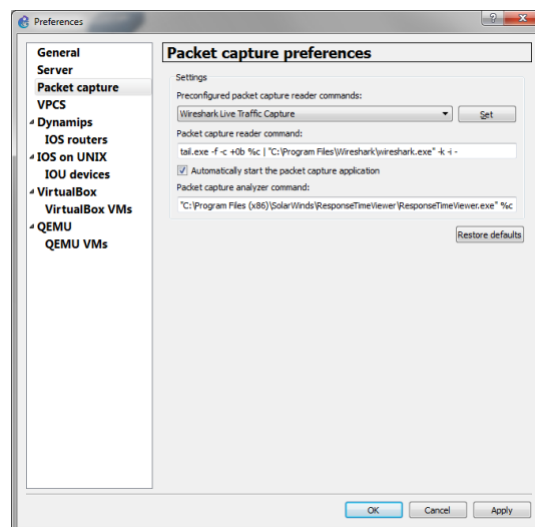


Figure 2.2 Packet capture Preferences

2. For Mac users the packet capture entries should be set as shown in Figure 2.3:

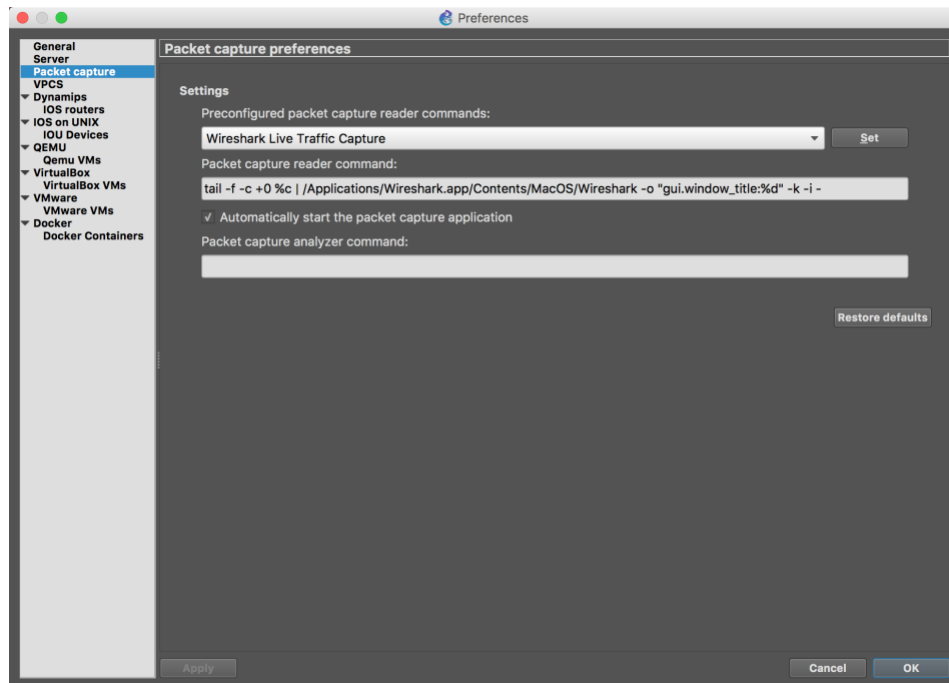


Figure 2.3 Mac OS Wireshark Settings

3. **Setting a display filter:** From the command “Display Filters...” under the “Analyze” menu, you can set a display filter so that only the traffic that matches the filter is displayed. Set a filter so that all packets that contain (in header, not data field) the IP address of PC2 (10.0.1.12) are displayed. Press “Enter/Return” after typing the filter.

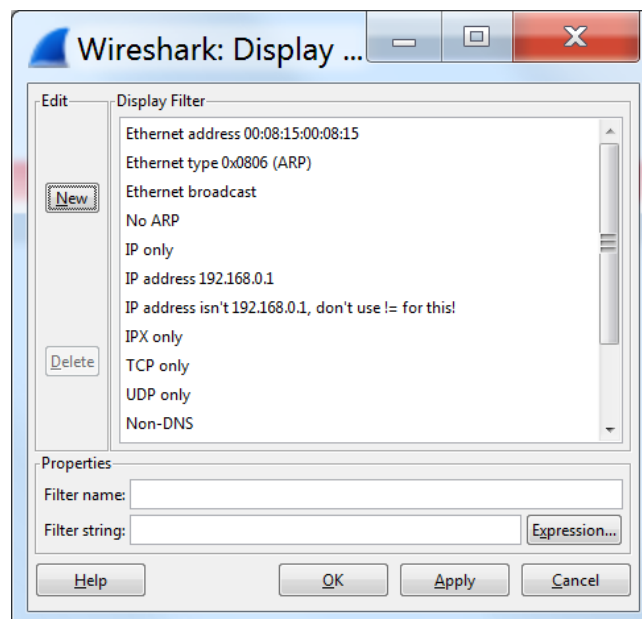


Figure 2.4 Display Filters command

4. You can also set a display filter by typing the desired display filter in the “Filter” box, which is found in the Wireshark main window as shown in Figure 2.5. Click the Clear button next to the filter box to clear any existing filter.



Figure 2.5 Filter box for setting display filters

5. In the terminal window of PC1, issue a ping command to PC2:

```
PC1% ping 10.0.1.12 -c 2
```

6. Stop the capture process.
7. **Saving captured traffic.** You can save the capture as a '.pcap' file by going to File -> Save. You can use this file for analysis at a later point by opening it in the Wireshark application. You can also save the results of Wireshark to a text file, using the "Packet details" option in "Print". One saved as a text file, you eliminate the option of viewing it in the Wireshark environment for later analysis. The drawback of that is that you will not be able to use any filters or other features of Wireshark to analyze the traffic.

PART 3. Address Resolution Protocol (ARP)

This part of the lab explores the operation of the Address Resolution Protocol (ARP) that resolves a MAC address for a given IP address. The lab exercises use the Linux command `arp`, for displaying and manipulating the contents of the ARP cache. The ARP cache is a table that holds entries of the form <IP address, MAC address>. The most common uses of the `arp` command are listed below.

COMMON USES OF THE ARP COMMAND

`arp -a`

Display the content of the ARP cache.

`arp -d IPaddress`

Deletes the cache entry with IP address **IPaddress**.

`arp -s IPaddress MACAddress`

Adds a static entry to the ARP cache that is never overwritten by network events. The MAC address is entered as 6 hexadecimal bytes separated by colons.

Example: `arp -s 10.0.1.12 00:02:2D:0D:68:C1`



TIME-OUTS IN THE ARP CACHE

The entries in an ARP cache have a limited lifetime. Entries are deleted unless they are refreshed. The typical lifetime of an ARP entry is 2 minutes, but much longer lifetimes (up to 20 minutes) have been observed.



FLUSHING THE ARP CACHE

You also can clear the ARP cache with the following command

```
ip -s -s neigh flush all
```



REFRESHING THE ARP CACHE

In Linux you will observe that a host occasionally sends out ARP requests to interfaces that are already in the ARP cache.

Example: Suppose that a host with IP address 10.0.1.22 has an ARP cache entry: 10.0.1.11 is at 08:00:27:53:63:1a

Then, this host occasionally sends a unicast ARP Request to MAC 08:00:27:53:63:1a of the form:

who has 10.0.1.11? Tell 10.0.1.22

to verify that the IP address 10.0.1.11 is still present/live before deleting the entry from the ARP cache when the timer times out. A response from 10.0.1.11 will refresh the timer.

Exercise 3(A). A simple experiment with ARP

1. On PC1, view the ARP cache with `arp -a` and delete all entries with the `-d` option.

```
PC1% arp -a
```

```
PC1% arp -d
```

2. Start Wireshark on the PC1-Hub1 link.
3. Issue a ping command from PC1 to PC2:

```
PC1% ping 10.0.1.12 -c 2
```

Observe the ARP packets in the Wireshark window. Explore the MAC addresses in the Ethernet headers of the captured packets.

Direct your attention to the following fields:

- The destination MAC address of the ARP Request packets.
 - The Type Field in the Ethernet headers of ARP packets and ICMP messages.
4. View the ARP cache again with the command `arp -a`. Note that ARP cache entries can get refreshed/deleted fairly quickly (~2 minutes).

```
PC1% arp -a
```

5. Save the results of Wireshark. You will use your Wireshark output to answer the questions below.

Lab Questions

- What is the destination MAC address of an ARP Request packet?
- What are the different Type Field values in the Ethernet headers that you observed?
- Use the captured data to analyze the process by which ARP acquires the MAC address for IP address 10.0.1.12.
- Why are ARP Request packets not transmitted (i.e. not encapsulated) as IP packets?
Because it has its own identifier

Exercise 3(B). Matching IP addresses and MAC addresses

Identify the MAC addresses of all the interfaces connected to the network and enter them in Table 2.2. You can obtain the MAC addresses from the ARP cache of a PC by issuing a `ping` command from that host to every other host on the network. Alternatively, you can obtain the MAC addresses from the output of the `ifconfig` command in the console window of each PC.

PCs	IP Address of eth0	MAC address of eth0
PC1	10.0.1.11 / 24	66:74:98:f8:9b:b3
PC2	10.0.1.12 / 24	e2:c7:05:25:64:c3
PC3	10.0.1.13 / 24	66:66:a5:09:1e:1e
PC4	10.0.1.14 / 24	8a:e4:56:0d:31:41

Table 2.2. IP and MAC addresses

Exercise 3(C). ARP requests for a non-existing address

Observe what happens when an ARP request is issued for an IP address that does not exist in the local subnet.

1. Start Wireshark on PC1-Hub1 link with a capture filter set to capture packets that contain the IP address of PC1.
2. Issue a ping command from PC1 to 10.0.1.22. (Note that this address does not exist in this network configuration.)

```
PC1% ping 10.0.1.22 -c 10
```

3. Save the captured output.

Lab Questions

- Using the saved output, describe the time interval between each ARP Request packet issued by PC1. Observe the method used by ARP to determine the time between retransmissions of an unsuccessful ARP Request.

PART 4. The NETSTAT Command

The Linux command `netstat` displays information on the network configuration and activity of a Linux system, including network connections, routing tables, interface statistics, and multicast memberships. The following exercise explores how to use the `netstat` command to extract different types of information about the network configuration of a host. This list shows four important uses of the `netstat` command.

`netstat -i`

Displays a table with statistics of the currently configured network interfaces.

`netstat -rn`

Displays the kernel routing table. The `-n` option forces `netstat` to print the IP addresses. Without this option, `netstat` attempts to display the host names.

`netstat -an`

`netstat -tan`

`netstat -uan`

Displays the active network connections. The `-a` option display all active network connections, the `-ta` option displays only information on TCP connections, and the `-tu` option displays only information on UDP traffic. Omitting the `-n` option prints host names, instead of IP addresses.

`netstat -s`

Displays summary statistics for each protocol that is currently running on the host.

Exercise 4. Using netstat commands

1. Display information on the network interfaces by typing

PC1% `netstat -in`

2. Display the content of the IP routing table by typing

PC1% `netstat -rn`

3. Display information on TCP and UDP ports that are currently in use by typing

PC1% `netstat -a`

4. Display the statistics of various networking protocols by typing

PC1% `netstat -s`

**NOTE**

The values of the statistics displayed by some of the `netstat` commands are reset each time a host is rebooted. Therefore, if you are doing this exercise immediately after rebooting the PC, the output of `netstat` may not be very useful.

Lab Questions

Using the `netstat` output, answer the following questions:

- What are the network interfaces of PC1 and what are the MTU (Maximum Transmission Unit) values of the interfaces? `netstat -in`
- Explain the role of interface `lo`, the loopback interface.
- In the `netstat -in` output, why are the values of RX-OK (packets received) and TX-OK (packets transmitted) different for interface `eth0` but identical for interface `lo` (the local loop)?

Role of `lo`:
Device identification
Routing information
Packet filtering

PART 5. More on Configuring IP Interfaces in LINUX

The `ifconfig` command, besides being used to configure parameters of network interfaces, such as assigning IP addresses it also includes the ability to enable and disable interfaces. The `ifconfig` command is usually run when a system boots up. In this case, the parameters are read from a file. Once the Linux system is running, the `ifconfig` command can be used to modify the network configuration parameters. This list shows how `ifconfig` is used to query the status of network interfaces and to enable and disable an interface.

`ifconfig`

Displays the configuration parameters of all active interfaces.

`ifconfig interface`

Displays the configuration parameters of a single **interface**. For example, `ifconfig eth0` displays information on interface `eth0`.

`ifconfig interface down`

Disables the interface. No traffic is sent or received on a disabled interface

`ifconfig interface up`

Enables an interface.

`ifconfig interface IPAddress/prefix`

e.g., `ifconfig eth0 10.0.1.8/24`

Assigns interface `eth0` the IP address `10.0.1.8` with prefix 24

Exercise 5. Changing the IP address of an interface

1. On PC4, run `ifconfig` and screenshot the output.
2. Change the IP address of interface `eth0` of PC4 to `10.0.1.11/24`.
3. Run `ifconfig` again and screenshot the output.



Tip: If you are not able to screenshot all the output on the screen (too much data), use the command `ifconfig interface` for *each* interface so that you can capture each one separately.

Lab Questions

- Explain the fields of the `ifconfig` output.

PART 6. DUPLICATE IP Addresses

In this part of the lab, you observe the effects of having more than one host with the same (duplicate) IP address in a network. After completing Exercise 5, the IP addresses of the Ethernet interfaces on the four PCs are as shown in table 2.3 below. Note that PC1 and P4 are assigned the same IP address.

PCs	IP Address of eth0
PC1	10.0.1.11 / 24
PC2	10.0.1.12 / 24
PC3	10.0.1.13 / 24
PC4	10.0.1.11 / 24

Table 2.3. IP addresses for Part 6

Exercise 6. Setting the same IP address on two hosts

1. Delete all entries in the ARP cache on all PCs.
2. Run Wireshark on PC3-Hub1 link to capture the network traffic to and from the duplicate IP address 10.0.1.11.
3. From PC3, issue a ping command to the *duplicate* IP address, 10.0.1.11, by typing

PC3% ping 10.0.1.11 -c 10
4. Stop Wireshark, save all ARP packets and screenshot the ARP cache of PC3 using the arp -a command:

PC3% arp -a
5. When you are done with the exercise, reset the IP address of PC4 to its original value as given in Table 2.1.

Lab Questions

- Explain how the ping packets were issued by the hosts with duplicate addresses.
- Did the ping command result in error messages?
- How can duplicate IP addresses be used to compromise the data security?
- Give an example. Use the ARP cache and the captured packets to support your explanation.